

HelixAgent

HelixAgent

Ne choisissez pas un seul modèle — laissez-les débattre, et livrez la réponse sur laquelle ils s'accordent.

Résumé

HelixAgent est un service d'ensemble LLM prêt pour la production, alimenté par AI, qui combine intelligemment les réponses de plusieurs modèles de langage — incluant un système de débat AI multi-tours et une sélection dynamique des fournisseurs basée sur la vérification — pour produire un résultat à la fois précis et fiable.

Description courte

HelixAgent est un service d'ensemble LLM basé sur Go qui fusionne plusieurs fournisseurs en une réponse unique et exacte. Il exécute des débats AI multi-tours, évalue dynamiquement les fournisseurs via LLMsVerifier, applique des stratégies de routage pondérées par la confiance, et intègre des fonctionnalités de production : mise en cache, supervision, garde-fous de sécurité et API de type OpenAI.

Description longue

HelixAgent est un service d'ensemble LLM prêt pour la production, alimenté par AI (sous licence MIT), qui considère la réponse d'un seul modèle comme une hypothèse, et non comme un verdict. Plutôt que de miser sur un unique fournisseur susceptible de se tromper, d'être biaisé ou temporairement indisponible, il combine les réponses de plusieurs modèles de langage pour converger vers le résultat le plus exact et fiable — et lorsque la question le justifie, il soumet les modèles à un débat structuré en plusieurs tours. Le panel est large : son fichier

README répertorie de nombreux fournisseurs LLM sous `internal/llm/providers/`, dont Claude, DeepSeek, Gemini, Mistral, Qwen et xAI/Grok.

Surtout, la sélection des fournisseurs n'est pas une liste de préférences statique : elle se mérite en temps réel. Les scores de vérification en direct, issus d'un LLMsVerifier intégré, pilotent le routage et assurent un repli élégant vers le fournisseur le plus performant, avec un rapport d'erreurs catégorisées en cas de dégradation. L'orchestrateur de débats AI transforme les désaccords en signal : il prend en charge plusieurs topologies (maillage, étoile, chaîne) et un protocole de phase rigoureux — Proposition → Critique → Revue → Synthèse — avec un apprentissage inter-débats pour que le système s'améliore dans la réconciliation des modèles au fil du temps. Les stratégies de routage incluent la sélection pondérée par la confiance, le consensus par vote majoritaire et la détection d'intention sémantique, le tout avec des réponses en streaming en temps réel, livrées jeton par jeton plutôt qu'après stabilisation complète de l'ensemble.

Le service est conçu pour résister en production, et pas seulement pour briller en démonstration : PostgreSQL et Redis forment une couche de données hautement disponible, Prometheus/Grafana/OpenTelemetry fournissent des métriques, tableaux de bord et traçage, tandis que l'authentification JWT, la limitation de débit, un moteur de garde-fous et la détection PII encadrent l'ensemble des contrôles nécessaires à un déploiement réel. Il est organisé en une vingtaine de modules distincts (EventBus, Observabilité, Authentification, Stockage, VectorDB, Embeddings, RAG, Mémoire, MCP, etc.), chacun traitant une préoccupation séparée, et propose un framework d'optimisation LLM (mise en cache sémantique, sortie structurée, streaming amélioré) avec des intégrations pour SGLang, LlamaIndex, LangChain, Guidance et LMQL. Comme les points de terminaison de complétion et d'ensemble sont compatibles avec OpenAI, un client existant peut pointer vers HelixAgent et bénéficier d'un raisonnement d'ensemble sans réécriture.

Contenu

Pourquoi nous l'avons conçu

Un seul modèle LLM peut se tromper, être biaisé ou indisponible. HelixAgent a été créé pour permettre aux applications de consulter plusieurs modèles simultanément, d'évaluer leurs réponses en fonction de leur fiabilité mesurée, et de basculer en douceur – transformant une dépendance fragile à un seul fournisseur en un ensemble résilient et auto-évalué.

Pourquoi c'est un changement de paradigme

Il industrialise le consensus multi-modèles – passant de « interroger plusieurs modèles et réconcilier leurs réponses » via des scripts ad hoc à un service de production. Au lieu de coder en dur un seul fournisseur en espérant le meilleur, les équipes bénéficient d'un routage guidé par des scores de vérification en temps réel, d'un protocole de débat structuré pour les questions nécessitant plus qu'une seule tentative, et d'une résilience de niveau production (une couche de données haute disponibilité, une observabilité complète et des garde-fous), le tout derrière une interface compatible OpenAI et API. L'avantage décisif ? Une adoption sans perturbation : une dépendance fragile à un seul fournisseur devient un ensemble résilient et auto-évalué, et les clients existants y passent en modifiant simplement un point de terminaison plutôt que leur code.

Ce qui est innovant

- Un débat structuré en plusieurs tours AI qui traite les désaccords entre modèles comme une ressource : topologies sélectionnables (maillage, étoile, chaîne), un protocole discipliné Proposition → Critique → Revue → Synthèse, et un apprentissage inter-débats qui se renforce avec le temps.
- Une sélection dynamique des fournisseurs basée sur des scores LLMsVerifier en temps réel, plutôt qu'une liste de préférences statique – l'ensemble oriente les requêtes vers celui qui performe réellement à l'instant T, et bascule élégamment en cas de défaillance.
- Un cadre natif d'optimisation Go pour les modèles LLM (cache sémantique, sortie structurée, streaming amélioré), autonome mais compatible avec des optimiseurs externes optionnels (SGLang, LlamaIndex, LangChain, Guidance, LMQL) si besoin, sans obligation.
- Une architecture modulaire composée d'une vingtaine de modules distincts, garantissant une séparation claire des préoccupations et ouvrant la voie à des fonctionnalités Big Data comme la mémoire distribuée et le streaming de graphes de connaissances.

Principaux défis techniques et nos solutions

- **Choisir parmi de nombreux fournisseurs inégaux.** Les fournisseurs varient en qualité et évoluent dans le temps, rendant toute classification fixe obsolète dès le lendemain.

Nous avons résolu ce problème en rendant la sélection continuellement mesurée : les scores LLMsVerifier alimentent un routage pondéré par la confiance et par vote majoritaire, avec un basculement fluide pour contourner un fournisseur en dégradation plutôt que de lui faire confiance aveuglément.

- **Obtenir une réponse fiable à des questions réellement complexes.** Un modèle unique, interrogé une seule fois, n'a aucun mécanisme pour détecter ses propres erreurs. L'Orchestrateur de Débat en fournit un - un débat multi-topologies et en phases (Proposition → Critique → Revue → Synthèse) qui force les modèles à se challenger et à affiner leurs réponses avant qu'une réponse finale ne soit synthétisée.
- **Faire fonctionner un ensemble en production, pas seulement dans un notebook.** Interroger plusieurs fournisseurs multiplie les points de défaillance potentiels. Nous les avons maîtrisés grâce à une couche de données haute disponibilité PostgreSQL+Redis, une observabilité Prometheus/Grafana/OpenTelemetry pour identifier les comportements anormaux d'un fournisseur ou d'un routage, et un périmètre de sécurité incluant une authentification JWT, une limitation de débit, un moteur de garde-fous et une détection PII.

Contenu

Pile technologique

- **Go** — choisi car le fait de répartir une seule requête entre plusieurs fournisseurs simultanément correspond exactement à l'usage des goroutines, et le déploiement en binaire unique permet de conserver le service (composé d'environ 20 modules) simple à livrer ; il constitue la base de l'ensemble du service et de chaque module interne.
- **Gin (Web API)** — choisi pour offrir une interface HTTP rapide et peu gourmande en ressources ; il sert les points de terminaison /v1 compatibles avec OpenAI pour les complétions, les discussions, le streaming et les ensembles, permettant aux clients existants d'adopter l'ensemble sans modification.
- **PostgreSQL** — choisi comme stockage durable pour les sessions, les analyses et les enregistrements de débats, afin que les décisions consensuelles et l'historique des débats soient vérifiables ; il ancre la couche de données haute disponibilité.
- **Redis** — choisi pour le cache à faible latence et la mise en file d'attente des tâches ; il alimente à la fois le cache des réponses et la couche de cache sémantique, permettant aux requêtes répétées ou quasi identiques d'éviter des inférences redondantes.

- **LLMsVerifier (intégré)** — choisi pour faire de la fiabilité des fournisseurs une mesure quantifiable plutôt qu’une simple hypothèse ; ses scores classent les fournisseurs pour le routage et déclenchent des solutions de repli en cas de dégradation.
- **Prometheus + Grafana + OpenTelemetry** — choisis pour garantir l’observabilité d’un ensemble couvrant de nombreux fournisseurs ; ils exposent les métriques `helixagent_*`, les tableaux de bord et le traçage des requêtes de bout en bout à travers la répartition des tâches.
- **Adaptateurs Model Context Protocol (MCP)** — choisis pour leur extensibilité via un protocole ouvert ; le README répertorie de nombreux adaptateurs MCP permettant de connecter des outils externes et des contextes.
- **Neo4j / ClickHouse / Kafka (BigData)** — choisis pour dépasser les limites d’un seul nœud : Neo4j et ClickHouse supportent la mémoire distribuée et les fonctionnalités de graphe de connaissances, tandis que Kafka diffuse ce graphe et ces données d’événements à grande échelle.
- **Intégrations d’optimisation (SGLang, LlamaIndex, LangChain, Guidance, LMQL)** — choisies pour ajouter en option des services tels que le cache de préfixes, la récupération, la décomposition des tâches et la génération contrainte, permettant ainsi de bénéficier d’optimisations plus poussées sans les rendre obligatoires.

Statut et notes de transparence

- **Statut : bêta.** Le service est présenté comme prêt pour la production, mais les chiffres de performance et de couverture mentionnés dans le README (par exemple, « 1000+ requêtes/seconde », « <500 ms en cache », nombre de fournisseurs et de scripts de validation) sont des affirmations auto-déclarées par le projet, non vérifiées de manière indépendante, et sont volontairement maintenues ici sous une forme qualitative.
- Le nombre de fournisseurs varie dans le README lui-même ; la page utilise une formulation qualitative du type « de nombreux fournisseurs ».

Niveau de priorité : Helix-primaire.